# Compiler Construction

Teacher : Zulfiqar Ali

Email: Zulfiqar.rafique@nu.edu.pk

Office: No.6,Basement -1 (CS Block 1)

# Week 2:

- Role of Lexical Analyser
- Input Buffering
- Specification and recognition of tokens
- Regular Expressions
- Finite Automata (DFA & NFA)

# Role of Lexical Analyser

- Read the Input characters
- Group them in Lexemes
- Produce sequence of Tokens as Output
- The stream of tokens is sent to the parser for syntax analysis.
- Interact with the symbol table
- When any identifier constitute put its information in symbol table
- Stripping out comments and whitespace (blank, newline, tab, and may be some other characters)
- Generate Errors which relates to Lexical Analysis.
- Lexical Analyzer may keep track record of line numbers to associate any error

# Input Buffering

- **Challenge in Lexeme Recognition:**
  - Reading source code efficiently is tricky because lexemes often require lookahead (checking extra characters) to confirm boundaries.
  - Why Lookahead is Needed:
  - Identifiers: End only when a non-alphanumeric character (e.g., space, operator) is encountered.
  - Multi-character Operators: Single chars like -, =, or < could be part of longer ops (e.g., ->, ==, <=).
  - Examples: x, iffy, &, &&

# **Solution: Two-Buffer Scheme**: (Buffer Pair)

- Handles large lookaheads safely by dividing input into **two buffers**, allowing seamless scanning.
- Two buffers (size = disk block, e.g., 4096 bytes) alternate to minimize system call.
- Special eof marker indicates end of file.
- **lexemeBegin:** Points to the start of the current lexeme.
- **forward:** Scans ahead to identifv lexeme boundaries.

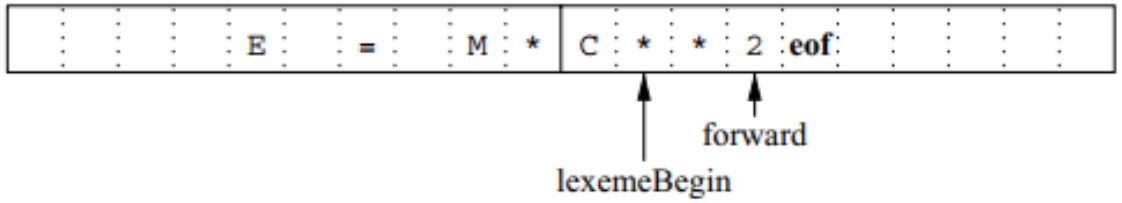
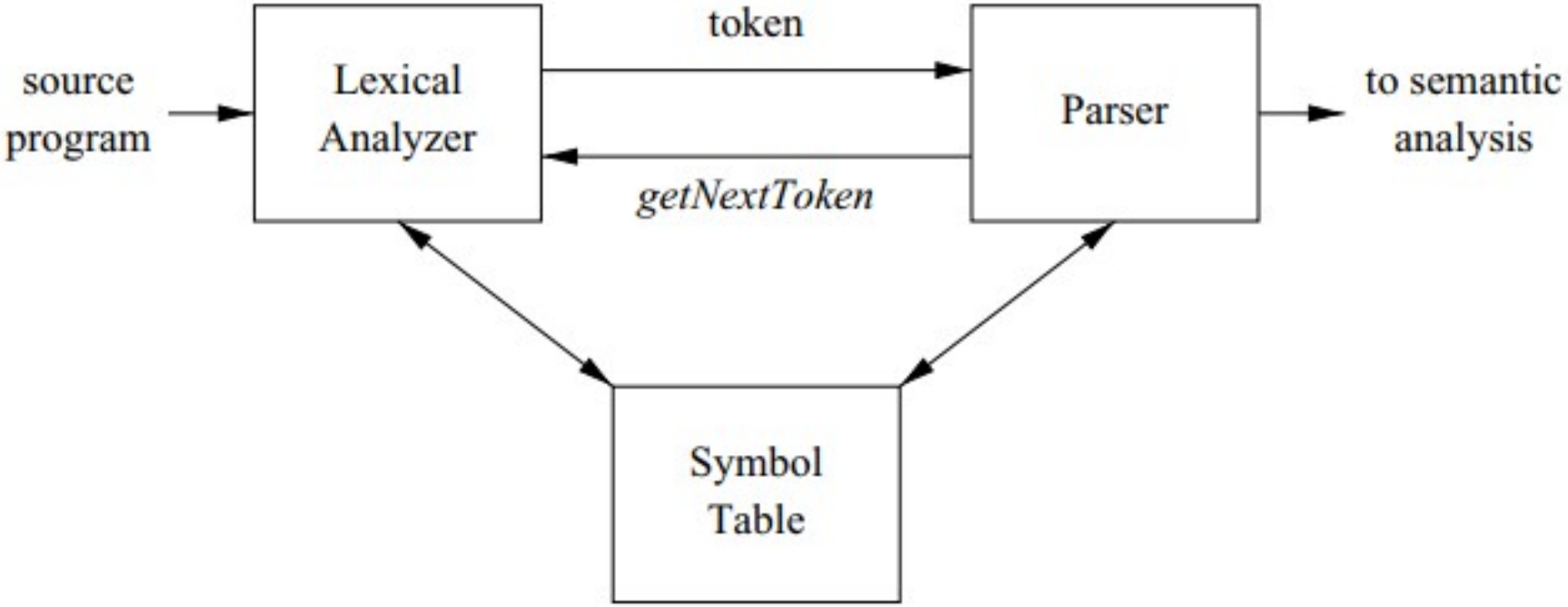



Figure 3.3: Using a pair of input buffers

# Interaction with Symbol Table



# Tokens, Patterns, and Lexemes

| Term | Definition | Example (int x = 10;) |
|---|---|---|
| **Lexeme** | Actual substring in code | "int", "x", "10" |
| **Pattern** | Rule defining valid lexemes | [a-z]+ (for identifiers) |
| **Token** | Categorized lexeme ((TYPE, "int")) | (INT, 10), (ID, "x") |

# Kinds of Tokens

- Most Languages have the tokens in these categories
  - Keywords – If, Else, Do, While
  - Identifiers - num, var, addfunction
  - Operators (+, -, *, /, <, >, <=)
  - Numbers/Constants 362, 457
  - Strings/Literals “This is literal”
  - Punctuators , “ “ ()[] ;
  - Special Characters & % ^ @

# Example of Tokenization

- Keywords:
- Identifiers:
- Operators:
- Strings/Literals:
- Punctuators:
- Constants:
- Special Characters:

```
int main()
{
  int x,a=2,b=3,c=5;
  x = a+b*c;
  printf("The value of x is %d",x);

  return 0;
}
```

# Exercises (Find the number of tokens)

- Printf(“i=%d, &i=%x” , I, &i);

```
int main)(
{
        in ta=32;
        a++;
        ch//comment ar d;
        printf("update values is %d", &a);
        a=b+++----+++==;
        printf("%d %d, a, b)
}
```

```
int main(){

        //Fining Max of a & B
        int a=20, b=30;
        if(a<b)
                return (b);
        else
                return (a);

}
```

# Symbol Table

- Symbol tables are data structures that are used by compilers to hold information about source-program constructs
- It allow the compiler to find the record for each name quickly and to store or retrieve data from that record quickly.
- The information is collected incrementally by the analysis phases of a compiler and used by the synthesis phases to generate the target code.

# Symbol Table Usage by Phases

| | | |
|---|---|---|
| **Symbol Table** | Lexical Analysis | Create entries for identifiers |
| | Syntax Analysis | Adds information regarding attributes(type, Scope, dimension etc) |
| | Semantic Analysis | Perform type checking, ensuring that expressions and assignments are semantically correct. |
| | Intermediate Code Generation | Available info helps in adding temporary variable information |
| | Code Optimization | Available info helps in perform various optimizations, such as identifying redundant computations or optimizing memory access patterns. |
| | Target Machine Code | The symbol table provides the memory addresses or register assignments for identifiers, which are crucial for generating the final machine code. |

# Data stored in the Symbol Table

Let's take a look at the kinds of information stored in the symbol table for different types of elements within a program.

| Lexeme | Token Class | Symbol Type | Data Type | Value | Scope |
|---|---|---|---|---|---|
| while | TokWhile | Keyword | None | 0 | Global |
| + | TokPlus | Operator | None | 0 | Global |
| x | TokIdentifier | Variable | Integer | 0 | Sample |
| 8 | TokConstant | Literal | Integer | 8 | Global |

# Errors

- Errors
  - Compiler Time Errors
    - Lexical Errors
    - Syntax Errors
    - Semantic Errors
  - Run Time Errors

# Lexical Errors

- It is hard for a lexical analyzer to tell, without the aid of other components,
- **fi** ( a == f(x))
  - A lexical analyzer cannot tell whether **fi** is a misspelling of the keyword **if** or an undeclared function identifier.
  - Since **fi** is a valid lexeme for the identifier token type.
  - This error may be handled by Parser.

# Lexical Errors

- Identifiers that are too long (anidentifierrrrrrrr……..)
- Exceeding length of Numeric Constants (int i = 454654564654;)
- Ill-formed numeric constants (45$36)
- Illegal characters which are not available in language (char x[]="FAST NUCES";$)

# Lexical Errors Recovery

- **Panic Mode Recovery**
  - int ~~4FAST~~; characters will be skipped until semicolon (Punctuator)
  - For(i=0; i<=10;i++){ in any line if Lexical Error found then from that line all code will be skipped until any delimiter found, Like **}**
- **Transpose of Two Adjacent Characters**
  - itn -- > int, unoin- union
- **Inserting a missing character**
  - it num; error handler may add **n**, to make it **int**
- **Deleting an unknown or extra character**
  - intt num; error handle may remove extra t, int num;
- **Replace a character by another**
  - itt num to **int num**

# Regular Expressions

- Regular expressions are a convenient way to specify various simple sets of strings.
- They are of practical interest because they can specify the structure of the tokens used in a programming language.
- Regular expressions (REs) are a language for expressing patterns.

A **regular expression** $s$ is a string which denotes $L(s)$, a set of strings drawn from an alphabet $\Sigma$. $L(s)$ is known as the "language of $s$."

$L(s)$ is defined inductively with the following base cases:

- If $\mathtt{a} \in \Sigma$ then $\mathtt{a}$ is a regular expression and $L(a) = \{a\}$.
- $\epsilon$ is a regular expression and $L(\epsilon)$ contains only the empty string.

Then, for any regular expressions $s$ and $t$:

1. $s|t$ is a RE such that $L(s|t) = L(s) \cup L(t)$.
2. $st$ is a RE such that $L(st)$ contains all strings formed by the concatenation of a string in $L(s)$ followed by a string in $L(t)$.
3. $s^*$ is a RE such that $L(s^*) = L(s)$ concatenated zero or more times.

Rule #3 is known as the **Kleene closure** and has the highest precedence. Rule #2 is known as **concatenation**. Rule #1 has the lowest precedence and is known as **alternation**. Parentheses can be added to adjust the order of operations in the usual way.

Here are a few examples using just the basic rules. (Note that a finite RE can indicate an infinite set.)

| **Regular Expression s** | **Language L(s)** |
|---|---|
| `hello` | { `hello` } |
| `d(o\|i)g` | { `dog,dig` } |
| `moo*` | { `mo,moo,mooo,...` } |
| `(moo)*` | { $\epsilon$,`moo,moomoo,moomoomoo,...` } |
| `a(b\|a)*a` | { `aa,aaa,aba,aaaa,aaba,abaa,...` } |

`s?` indicates that `s` is optional.
`s?` can be written as `(s|`$\epsilon$`)`

`s+` indicates that `s` is repeated one or more times.
`s+` can be written as `ss*`

`[a-z]` indicates any character in that range.
`[a-z]` can be written as `(a|b|...|z)`

`[^x]` indicates any character except one.
`[^x]` can be written as $\Sigma$ - x

| | |
|---|---|
| **Associativity:** | `a|(b|c) = (a|b)|c` |
| **Commutativity:** | `a|b = b|a` |
| **Distribution:** | `a(b|c) = ab|ac` |
| **Idempotency:** | `a** = a*` |

| | |
|---|---|
| Informal definition: | *An identifier is a sequence of capital letters and numbers, but a number must not come first.* |
| Regular expression: | `[A-Z]+([A-Z]\|[0-9])*` |
| Matches strings: | `PRINT`<br>`MODE5` |
| Does not match: | `hello`<br>`4YOU` |

| | |
|---|---|
| Informal definition: | *A number is a sequence of digits with an optional decimal point. For clarity, the decimal point must have digits on both left and right sides.* |
| Regular expression: | `[0-9]+(.[0-9]+)?` |
| Matches strings: | `123`<br>`3.14` |
| Does not match: | `.15`<br>`30.` |

| | |
|---|---|
| Informal definition: | *A comment is any text (except a right angle bracket) surrounded by angle brackets.* |
| Regular expression: | `<[^>]*>` |
| Matches strings: | `<tricky part>`<br>`<<<<look left>` |
| Does not match: | `<this is an <illegal> comment>` |

# Regular definition (Pattern Representation)

- Regular expressions are used to represent the patterns for the tokens.

The regular definition for **C** identifiers (**letters, digits,** and **underscores**).

- *Letter* → A | B | C|...| Z | a | b | … |z| _
- *digit* → *0|1|2* |... | *9*
- *id* → *letter*( *letter* | *digit* )* E.g. a E.g. a_12b

The regular definition for **Unsigned numbers (integer** or **floating point)**:

- *digit* → *0|1|2* |... | *9* E.g. 9
- *digits* → *digit digit** E.g. 952
- *optionalFraction* → *.digits* | *ε* E.g. .23
- *optionalExponent* → ( E( + |- | *ε*) *digits* ) | *ε* E.g. E+2 E.g. E-2 E.g. E 2
- *number* → *digits optionalFraction optionalExponent*

- More examples: integer constant, string constants, reserved words, operator, real constant.

# Exercises for Regular Expressions

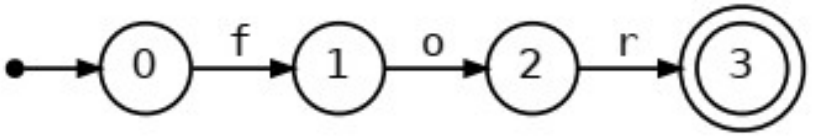
- Write a Regular Expression for Identifiers (Format: must start with a small alphabet, followed by number/alphabet/_)

- Write a regular Expression for Decimal Point Numbers

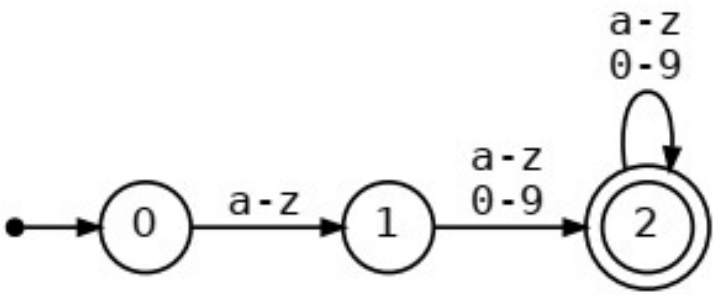
- Write a Regular Expression for Keywords : If, For, While

# Finite Automata

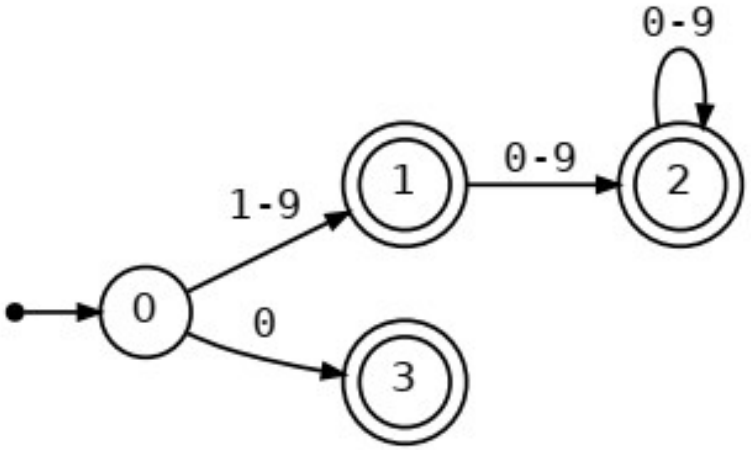
- A **finite automaton (FA)** is an abstract machine that can be used to represent certain forms of computation.
- Graphically, an FA consists of a number of states (represented by numbered circles) and a number of edges (represented by labelled arrows) between those states.
- Each edge is labelled with one or more symbols drawn from an alphabet Σ
- **Types**
  - Deterministic FA
  - Non-Deterministic FA




Here is an FA for identifiers of the form `[a-z][a-z0-9]+`




And here is an FA for numbers of the form `([1-9][0-9]*)|0`

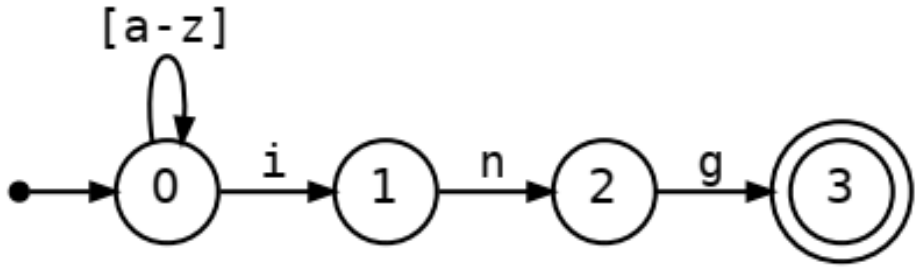


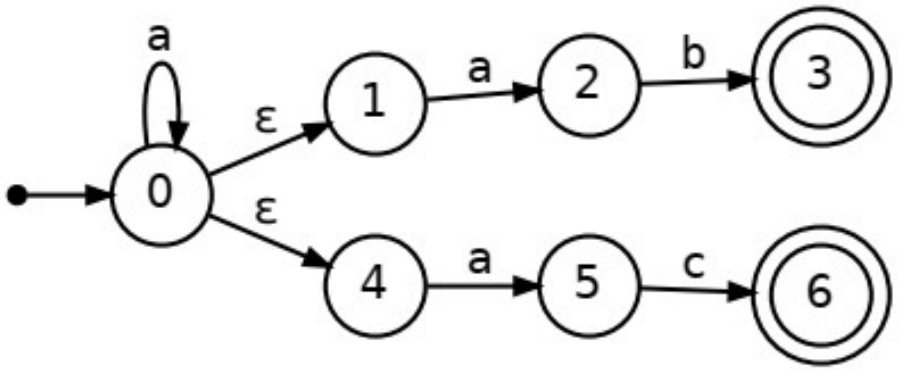
# DFA – Deterministic FA

- A DFA is a special case of an FA where every state has no more than one outgoing edge for a given symbol.
- A DFA has no ambiguity: for every combination of state and input symbol, there is exactly one choice of what to do next.
- Because of this property, a DFA is very easy to implement in software or hardware.

# Non-Deterministic FA

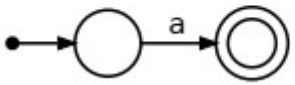
- An NFA is a perfectly valid FA, but it has an ambiguity that makes it somewhat more difficult to work with.
- Consider the regular expression [a-z]*ing, which represents all lowercase words ending in the suffix ing.



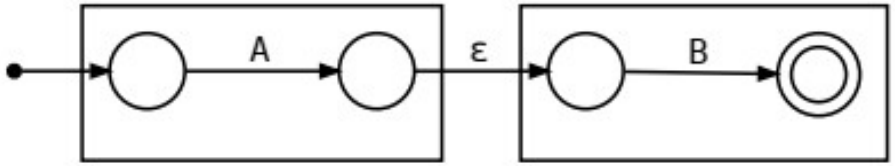
# Converting RE's to NFA's

The NFA for any character a is:

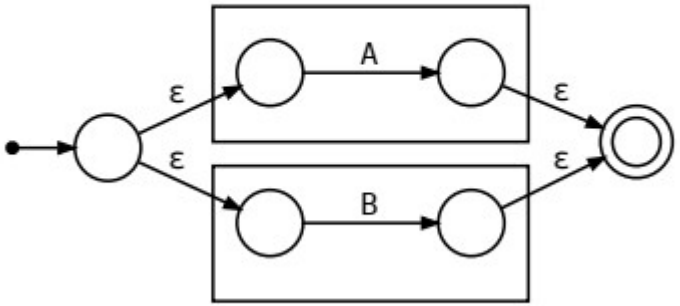



The NFA for an $\epsilon$ transition is:

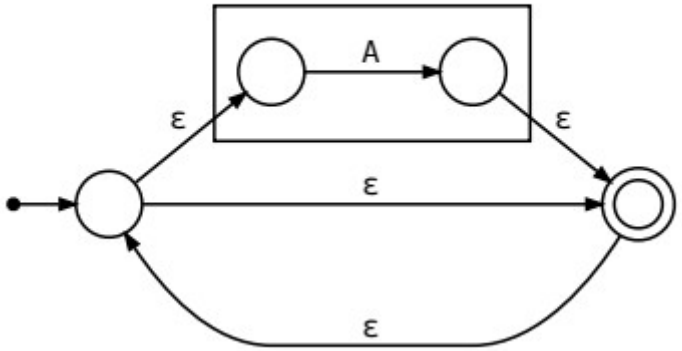
The NFA for the concatenation AB is:



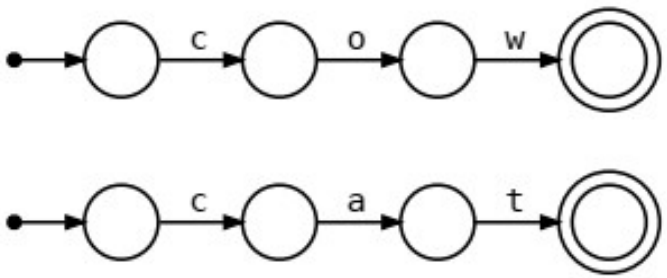

The NFA for the alternation A | B is:

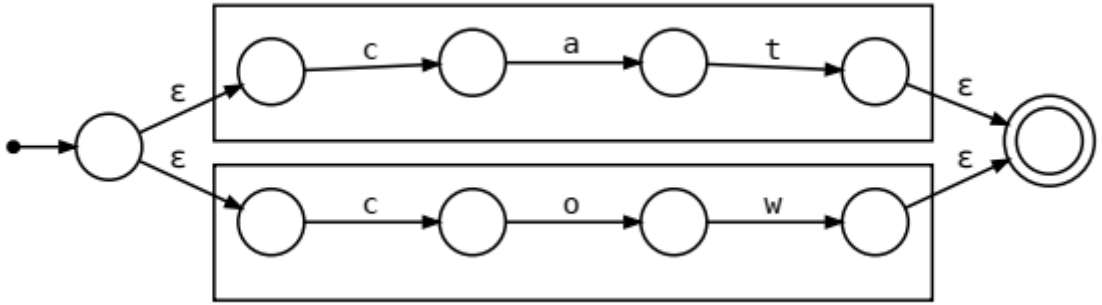



The NFA for the Kleene closure A* is:

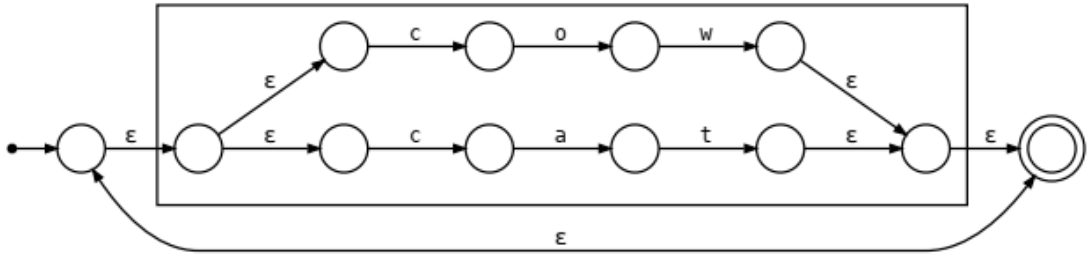


# Converting RE's to NFA's

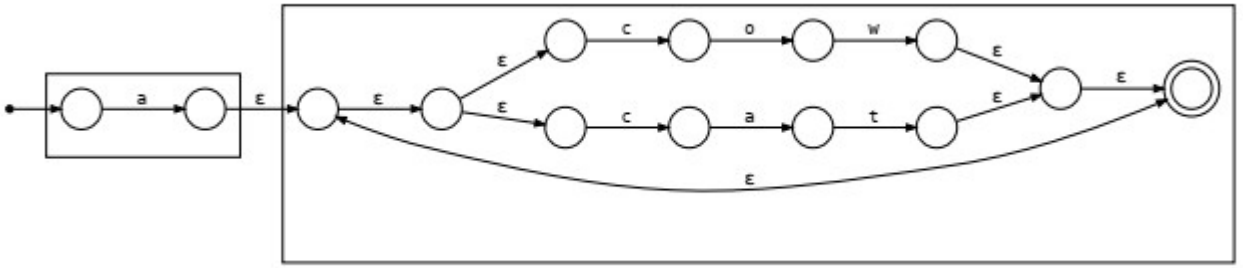



Kleene closure `(cat|cow)*`




concatenation of `a(cat|cow)*`



# Converting NFA's to DFA's (Subset Construction)

**Epsilon closure.**
$\epsilon-\text{closure}(n)$ is the set of NFA states reachable from NFA state $n$ by zero or more $\epsilon$ transitions.

**Subset Construction Algorithm.**
Given an NFA with states $N$ and start state $N_0$, create an equivalent DFA with states $D$ and start state $D_0$.

Let $D_0 = \epsilon-\text{closure}(N_0)$.
Add $D_0$ to a list.
While items remain on the list:
    Let $d$ be the next DFA state removed from the list.
    For each character $c$ in $\Sigma$:
        Let $T$ contain all NFA states $N_k$ such that:
            $N_j \in d$ and $N_j \xrightarrow{c} N_k$
        Create new DFA state $D_i = \epsilon-\text{closure}(\text{T})$
        If $D_i$ is not already in the list, add it to the end.

**Figure 3.3: Subset Construction Algorithm**

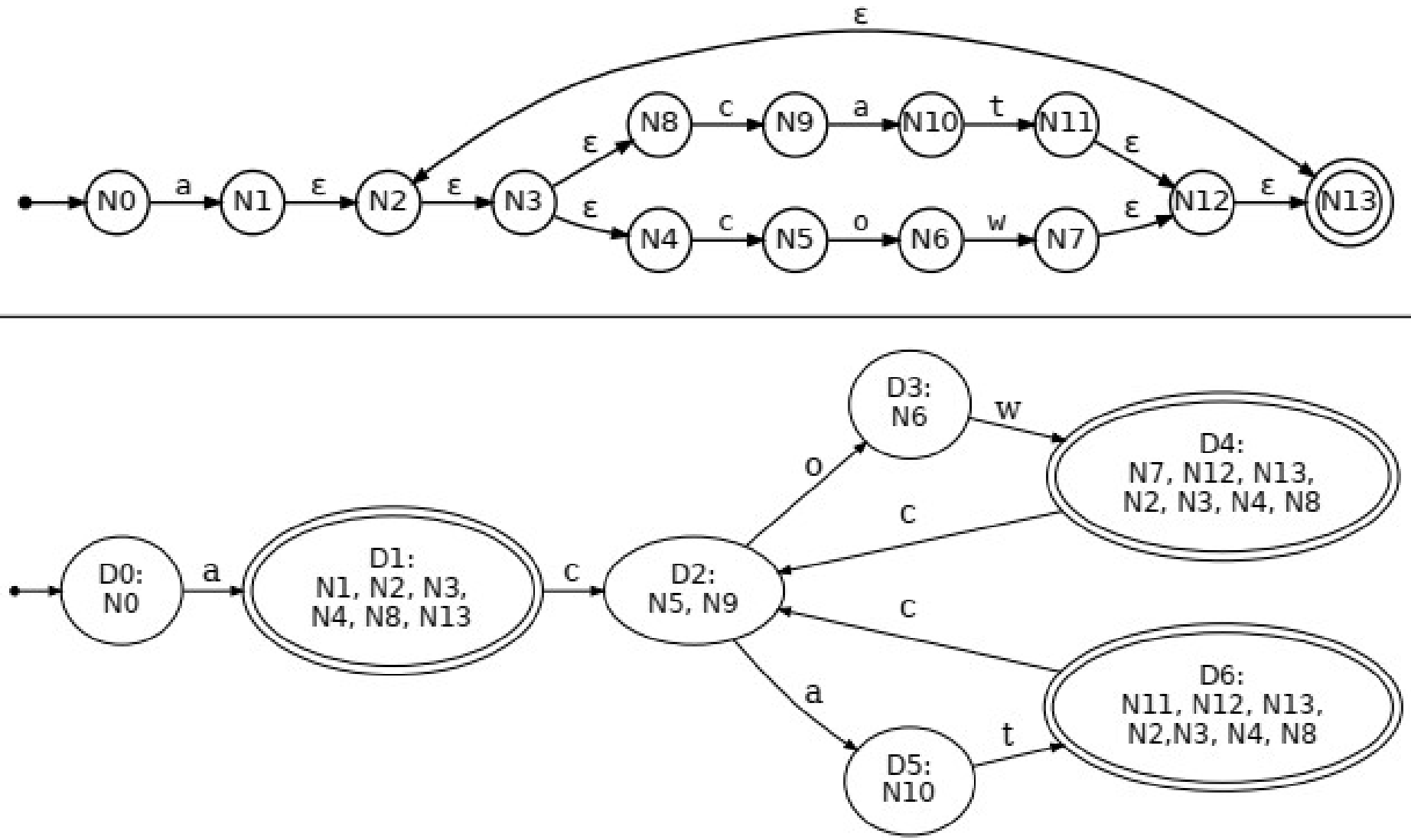



**Figure 3.4: Converting an NFA to a DFA via Subset Construction**

# Token Recognition

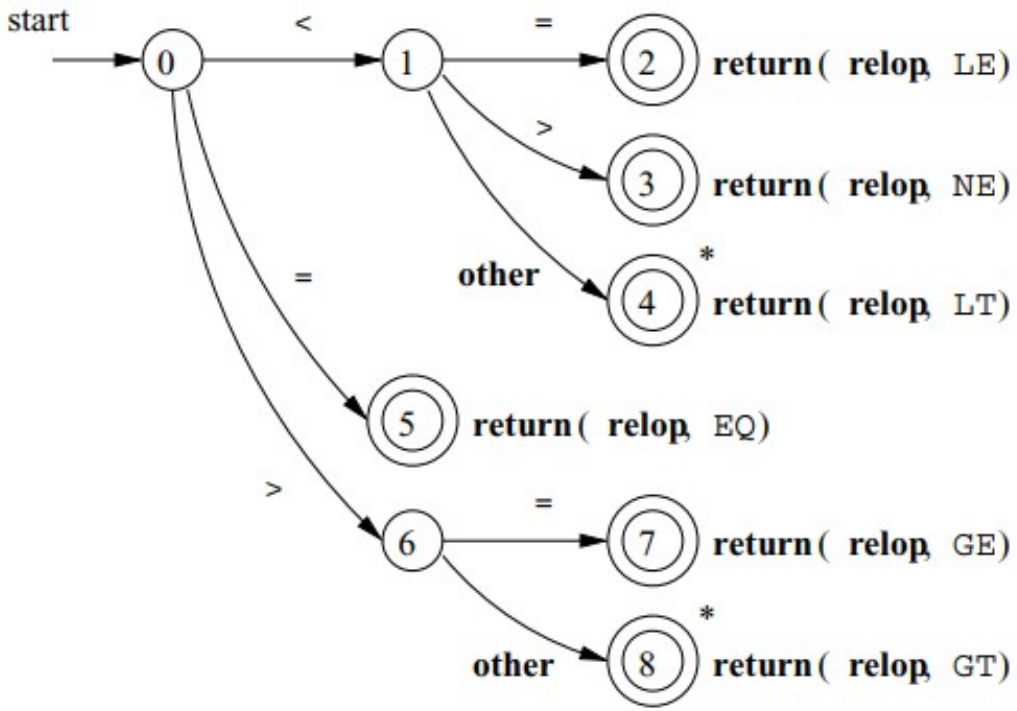
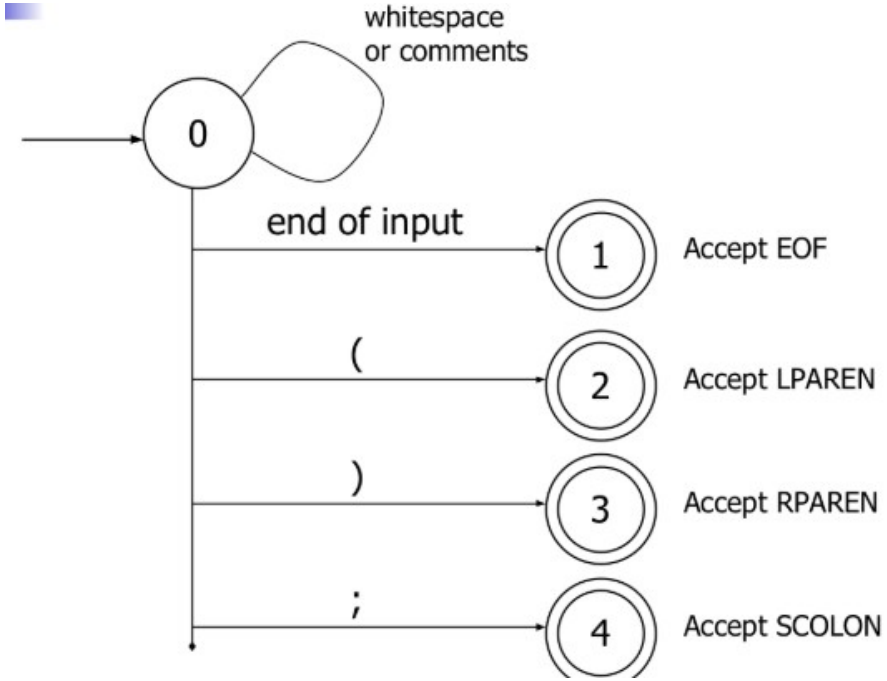
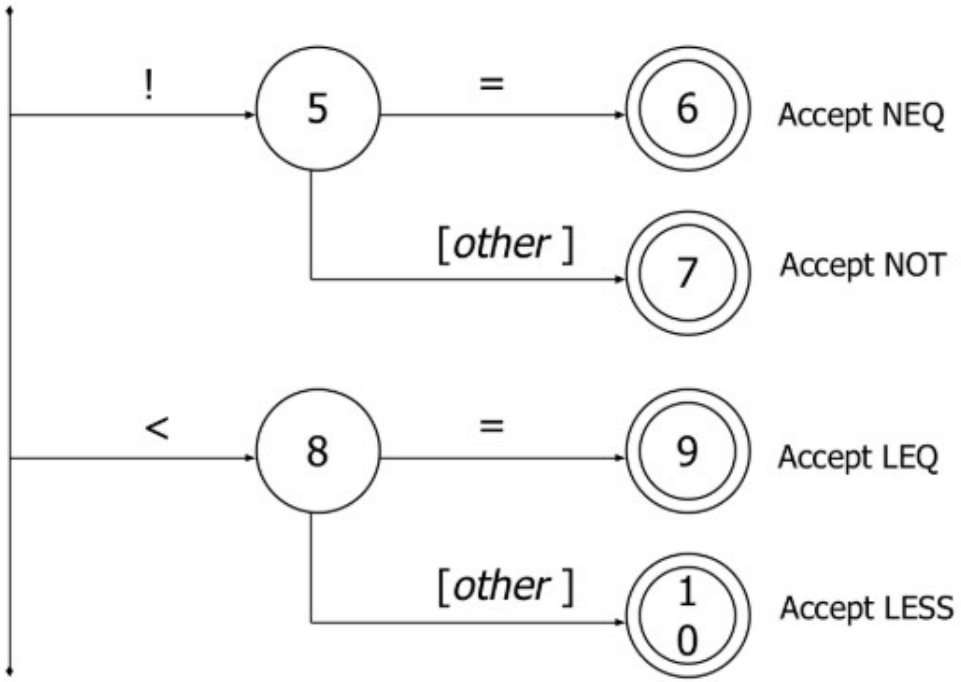
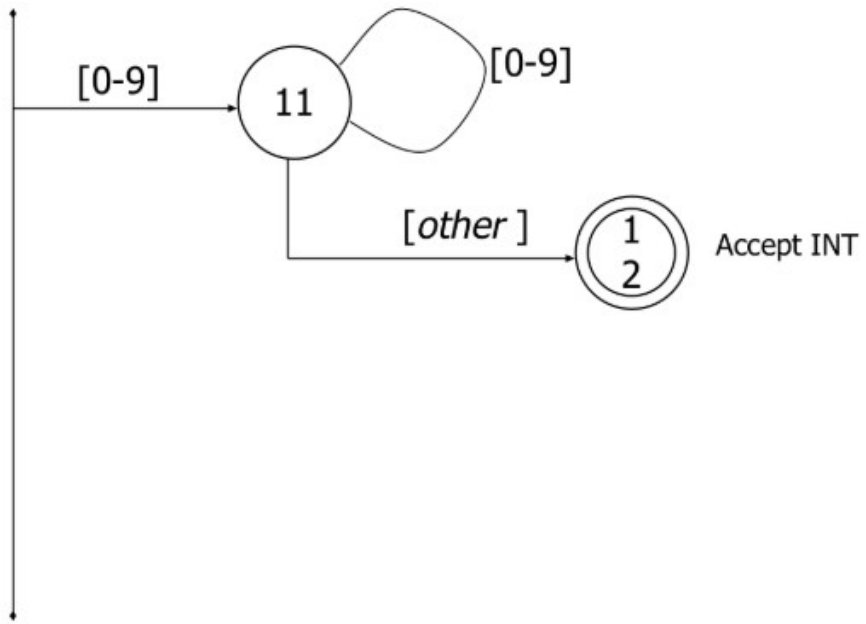
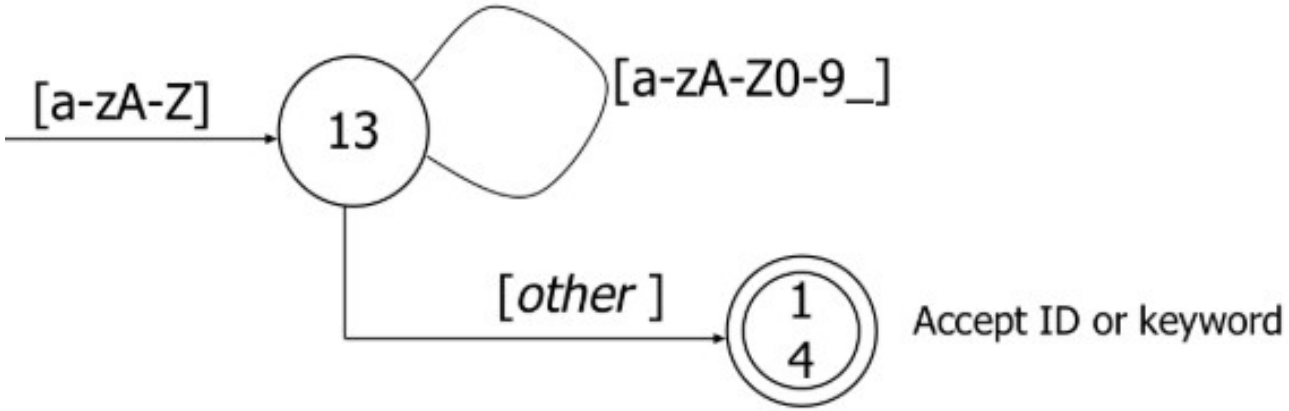



Figure 3.13: Transition diagram for **relop**


whitespace
or comments
0
end of input
1
Accept EOF
(
2
Accept LPAREN
)
3
Accept RPAREN
;
4
Accept SCOLON

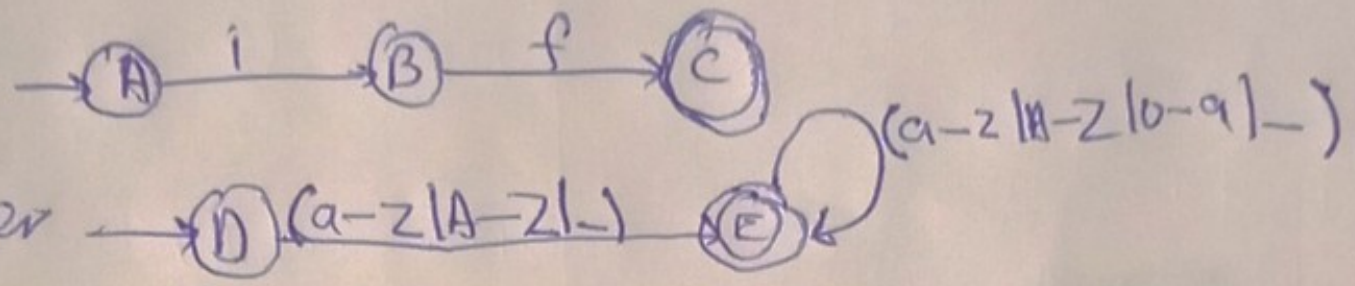

!
5
=
6
Accept NEQ
[other ]
7
Accept NOT
<
8
=
9
Accept LEQ
[other ]
10
Accept LESS


[0-9]
11
[0-9]
[other ]
12
Accept INT


[a-zA-Z]
13
[a-zA-Z0-9_]
[other ]
14
Accept ID or keyword

# Tokens

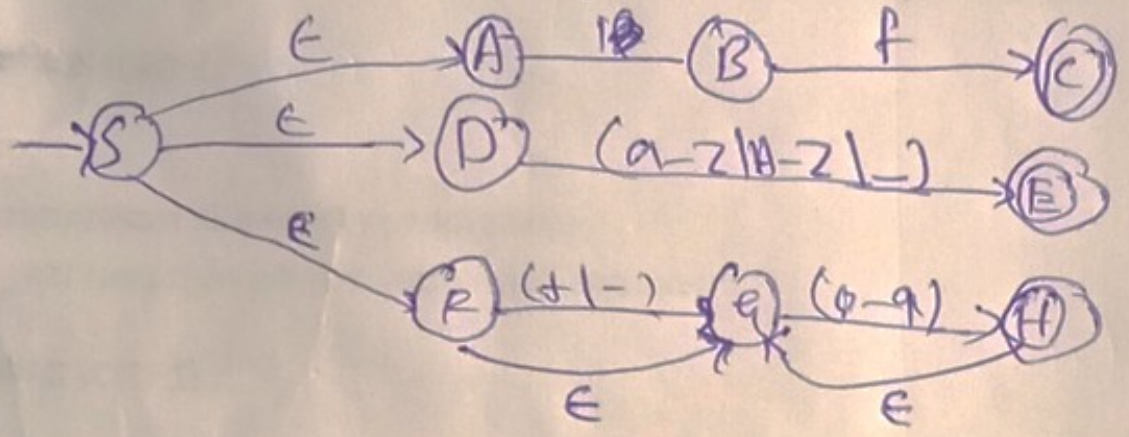
- if

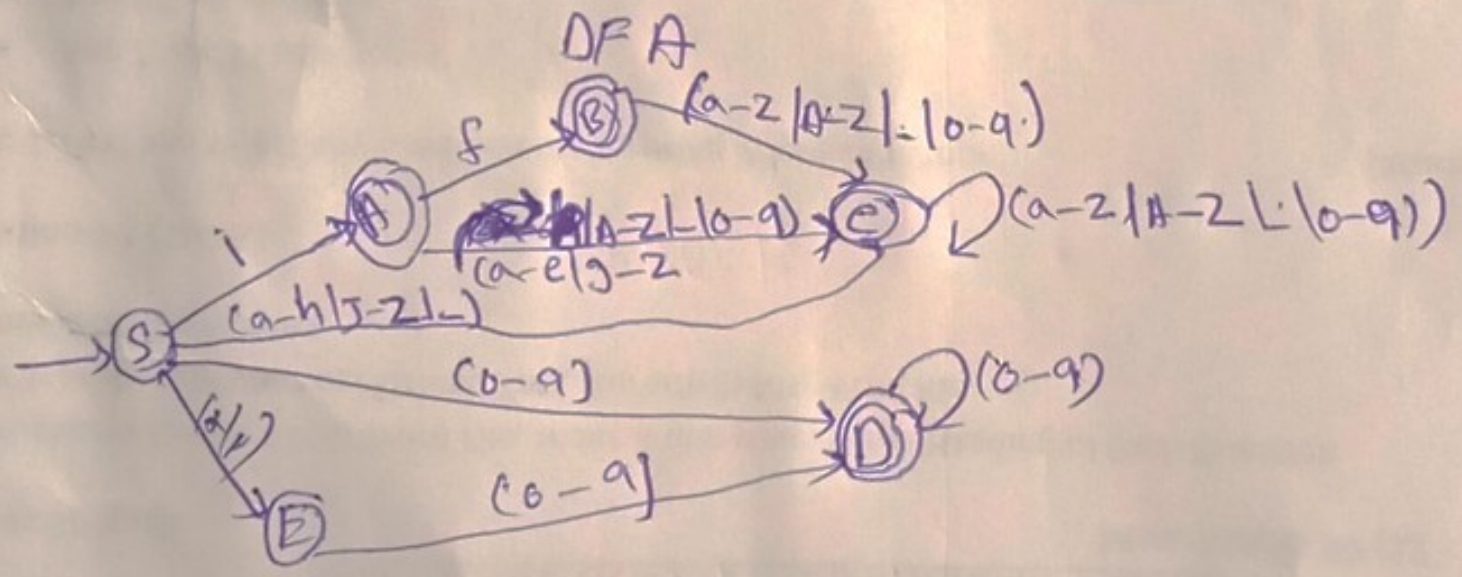



- identifier

- Integer

## NFA






## Lexemes

if ✓, iff ✓, iba ✓, institute ✓, nums, 647, -39 ✓, var ✓,

---

NFA ⟶ DFA —— minimized DFA

# Relational Operators Transition Diagram (TD)

Let us write TD that recognizes the lexemes matching the tokens <=, >=,>, < =, <=,>=,>, <, = and < >

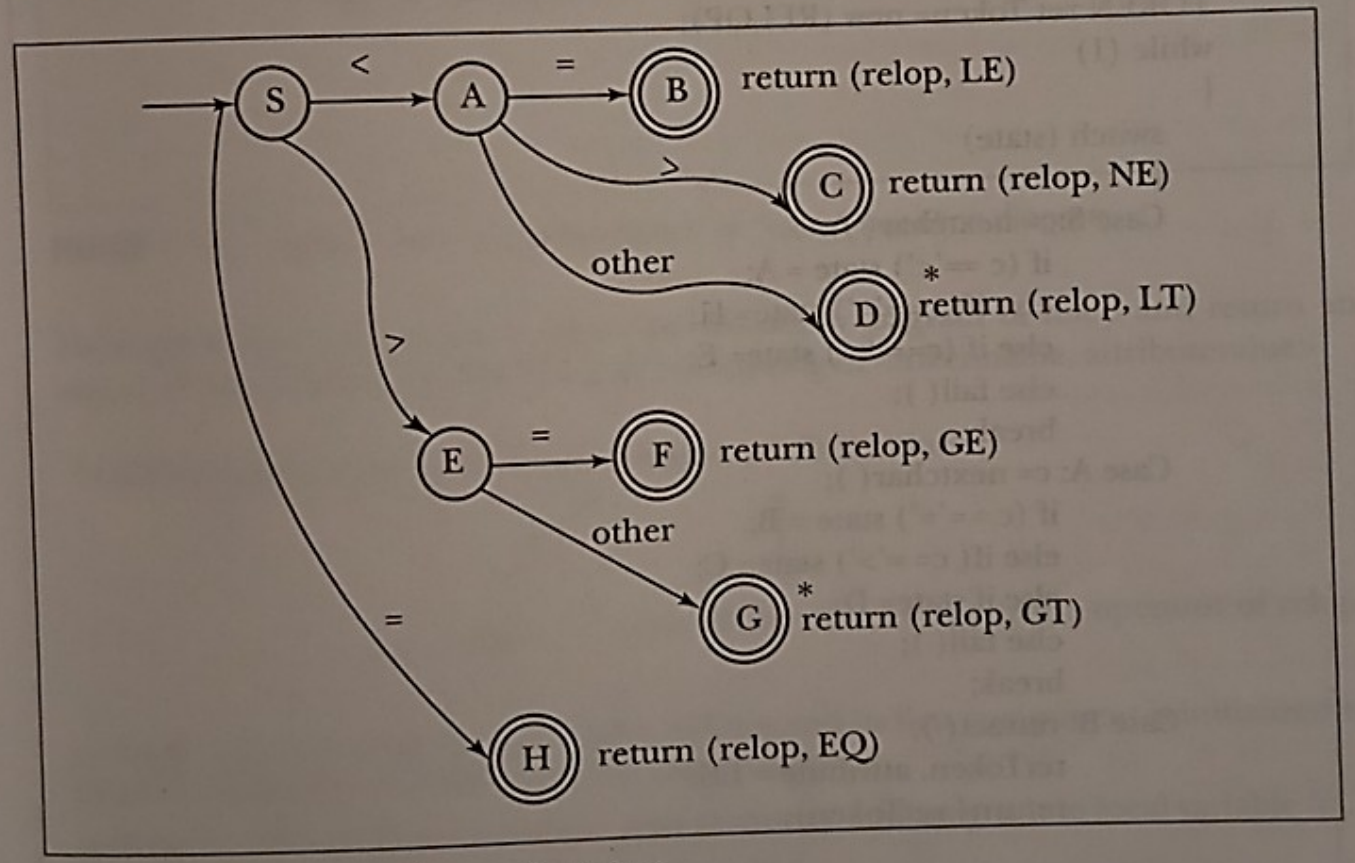



**FIGURE 1.13:** Transition diagram for 'relop'

# Relational Operators Program(For Lexical)

```
{
    TOKEN ret Token= new (RELOP);
    while (1)
    {
        switch (state)
        {
        Case S:c= nextchar( );
                if (c =='<') state = A;
                else if( c=='=') state= H;
                else if (c== '>') state= E;
                else fail( );
                break;
        Case A: c= nextchar( );
                if (c =='=') state = B;
                else if( c= ='>') state= C;
                else if state= D;
                else fail( );
                break;
        Case B: retract( );
                retToken. attribute = LE;
                return( retToken);
```

```
        Case C: retract( );
                retToken. attribute = NE;
                return( retToken);
        Case D:  retract( );
                retToken. attribute = LT;
                return( retToken);
        Case E: c = nextchar( );
                if (c =='=') state = F;
                else if  state = G;
                else fail( );
                break;
        Case F:  retract( );
                retToken. attribute = GE;
                return( retToken);
        Case G: retract( );
                retToken. attribute = GT;
                return( retToken);
        Case H: retract( );
                retToken. attribute = EQ;
                return( retToken);
        }
}
```

# Reference:

- Compilers: Principles, Techniques, and Tools 2nd Edition by Alfred Aho , Jeffrey Ullman, Ravi Sethi, Monica Lam

- Introduction to Compilers and Language Design | Second Edition | Prof. Douglas Thain - University of Notre Dame